Listing 5.20 Adding fields to a document using the `_update` API

```
POST movies/_update/1
{
  "doc": {
    "actors":["Marlon Brando","Al Pacino","James Caan"],
    "director":"Francis Ford Coppola"
  }
}
```

This query adds `actors` and `director` fields to our `movie` document. If we fetch the document (`GET movies/_doc/1`), the additional fields are available in the return document.

MODIFYING EXISTING FIELDS

Sometimes we may need to change existing fields. Doing so isn't complicated; we just have to provide the new value to the field in a `doc` object as we did in the previous query. For example, to rename our `title` field, we write a query like the one shown in the next listing.

Listing 5.21 Updating the title of an existing document

```
POST movies/_update/1
{
  "doc": {
    "title":"The Godfather (Original)"
  }
}
```

When it comes to updating an element in an array (like adding a new actor to the list in the `actors` field), we must provide both the new and old values. For example, let's say we want to add another actor (Robert Duvall) to the `actors` field for our *The Godfather* document.

Listing 5.22 Updating an existing field with additional information

```
POST movies/_update/1
{
  "doc": {
    "actors":["Marlon Brando",
              "Al Pacino",
              "James Caan",
              "Robert Duvall"]
  }
}
```

We're updating the document with ID 1.

The updates must be enclosed in the doc object.

Old and new values together

The query updates the `actors` field to add Robert Duvall. Note that we provide the existing actors in the array along with our new actor. If we only included Robert Duvall in the `actors` array, Elasticsearch would replace the list with just his name.